EECS 348 Assignment 4

GenAI Code Comparison

a. Name of two GenAI programs
ChatGPT (GPT-6)
Google Gemini

b. How you accessed them (e.g., Google query, CoPilot on my computer)
https://chatgpt.com/
https://gemini.google.com/

c. Prompt used.
Write a complete C++ program that solves 9x9 Sudoku puzzles using object-oriented programming and recursive depth-first search with backtracking. Use a class to represent the Sudoku solver. Read five text files named puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt. Each file contains a 9x9 Sudoku puzzle with digits 1-9 and underscores (_) representing empty cells. For each file, print the filename, original puzzle, and every valid solution if multiple solutions exist. If no solution exists, print "No solution found". Validate Sudoku rules for rows, columns, and 3x3 blocks. Put the entire program in a single C++ file. Use standard C++ libraries and make the program compatible with g++ on Linux.

d. Generated code from both GenAIs.

ChatGPT

```
#include <iostream>

#include <fstream>

#include <string>

using namespace std;


class SudokuSolver {

private:

    int grid[9][9]{};

    int solutionCount = 0;


    bool allowed(int row, int col, int value) const {

        for (int i = 0; i < 9; ++i) {

            if (grid[row][i] == value || grid[i][col] == value)

                return false;

        }

        int boxRow = (row / 3) * 3;

        int boxCol = (col / 3) * 3;
```

```
    for (int r = boxRow; r < boxRow + 3; ++r)
        for (int c = boxCol; c < boxCol + 3; ++c)
            if (grid[r][c] == value)
                return false;
    return true;
}

bool findEmpty(int &row, int &col) const {
    for (row = 0; row < 9; ++row)
        for (col = 0; col < 9; ++col)
            if (grid[row][col] == 0)
                return true;
    return false;
}

void printGrid() const {
    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c)
            cout << (grid[r][c] == 0 ? '_' : char('0' + grid[r][c])) << (c == 8 ? '\n' : ' ');
    }
}

void solveRecursive() {
    int row, col;
    if (!findEmpty(row, col)) {
        ++solutionCount;
        cout << "Solution " << solutionCount << ":\n";
        printGrid();
        return;
    }
    for (int value = 1; value <= 9; ++value) {
        if (allowed(row, col, value)) {
            grid[row][col] = value;
            solveRecursive();
```

```
                grid[row][col] = 0;
            }
        }
    }

public:
    bool load(const string &fileName) {
        ifstream file(fileName);
        if (!file)
            return false;
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                char ch;
                if (!(file >> ch) || (ch != '_' && (ch < '1' || ch > '9')))
                    return false;
                grid[r][c] = ch == '_' ? 0 : ch - '0';
            }
        }
        for (int r = 0; r < 9; ++r)
            for (int c = 0; c < 9; ++c)
                if (grid[r][c] != 0) {
                    int value = grid[r][c];
                    grid[r][c] = 0;
                    bool valid = allowed(r, c, value);
                    grid[r][c] = value;
                    if (!valid)
                        return false;
                }
        solutionCount = 0;
        return true;
    }

    void run(const string &fileName) {
        cout << "File: " << fileName << "\n";
        if (!load(fileName)) {
```

```
            cout << "Could not read a valid puzzle.\n\n";
            return;
        }
        cout << "Original puzzle:\n";
        printGrid();
        solveRecursive();
        if (solutionCount == 0)
            cout << "No solution found\n";
        cout << "\n";
    }
};

int main() {
    for (int i = 1; i <= 5; ++i) {
        SudokuSolver solver;
        solver.run("puzzle" + to_string(i) + ".txt");
    }
    return 0;
}
```

Gemini Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <array>

class SudokuSolver {
private:
    static const int SIZE = 9;
    static const int SUBGRID = 3;
    std::array<std::array<int, SIZE>, SIZE> board;
    std::vector<std::array<std::array<int, SIZE>, SIZE>> solutions;

    // Check if placing num at board[row][col] is valid
```

```
bool isValid(int row, int col, int num) const {
    for (int i = 0; i < SIZE; ++i) {
        // Check row and column
        if (board[row][i] == num || board[i][col] == num) {
            return false;
        }
    }

    // Check 3x3 subgrid
    int startRow = (row / SUBGRID) * SUBGRID;
    int startCol = (col / SUBGRID) * SUBGRID;
    for (int r = 0; r < SUBGRID; ++r) {
        for (int c = 0; c < SUBGRID; ++c) {
            if (board[startRow + r][startCol + c] == num) {
                return false;
            }
        }
    }

    return true;
}

// Verify initial input grid for conflicts before attempting to solve
bool isInitialStateValid() const {
    for (int r = 0; r < SIZE; ++r) {
        for (int c = 0; c < SIZE; ++c) {
            int val = board[r][c];
            if (val != 0) {
                // Temporarily empty cell to check against other filled cells
                const_cast<SudokuSolver*>(this)->board[r][c] = 0;
                bool valid = isValid(r, c, val);
                const_cast<SudokuSolver*>(this)->board[r][c] = val;
                if (!valid) return false;
            }
        }
```

```
    }
    return true;
}

// Recursive depth-first search with backtracking to find all solutions
void solveDFS(int row, int col) {
    // Reached end of board; store current complete solution
    if (row == SIZE) {
        solutions.push_back(board);
        return;
    }

    // Determine next cell coordinates
    int nextRow = (col == SIZE - 1) ? row + 1 : row;
    int nextCol = (col == SIZE - 1) ? 0 : col + 1;

    // Skip pre-filled cells
    if (board[row][col] != 0) {
        solveDFS(nextRow, nextCol);
        return;
    }

    // Try numbers 1 through 9
    for (int num = 1; num <= SIZE; ++num) {
        if (isValid(row, col, num)) {
            board[row][col] = num;       // Place digit
            solveDFS(nextRow, nextCol);  // Recurse
            board[row][col] = 0;         // Backtrack
        }
    }
}

// Utility function to render a 9x9 grid to an output stream
static void printBoard(const std::array<std::array<int, SIZE>, SIZE>& grid, std::ostream& os) {
```

```
        for (int r = 0; r < SIZE; ++r) {
            if (r > 0 && r % SUBGRID == 0) {
                os << "------+-------+------\n";
            }
            for (int c = 0; c < SIZE; ++c) {
                if (c > 0 && c % SUBGRID == 0) {
                    os << "| ";
                }
                if (grid[r][c] == 0) {
                    os << "_ ";
                } else {
                    os << grid[r][c] << " ";
                }
            }
            os << "\n";
        }
    }

public:
    SudokuSolver() {
        for (auto& row : board) {
            row.fill(0);
        }
    }

    // Load puzzle from input file stream
    bool loadFromFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        solutions.clear();
        char ch;
        int count = 0;
```

```
    while (file >> ch && count < SIZE * SIZE) {
        int r = count / SIZE;
        int c = count % SIZE;

        if (ch >= '1' && ch <= '9') {
            board[r][c] = ch - '0';
            count++;
        } else if (ch == '_') {
            board[r][c] = 0;
            count++;
        }
        // Non-matching characters (e.g. whitespace, newlines) are skipped
    }

    file.close();
    return count == SIZE * SIZE;
}

// Print original loaded board state
void printOriginal(std::ostream& os = std::cout) const {
    printBoard(board, os);
}

// Execute solver and display results
void solveAndDisplay(std::ostream& os = std::cout) {
    if (!isInitialStateValid()) {
        os << "No solution found (Initial puzzle contains invalid duplicate values).\n";
        return;
    }

    solveDFS(0, 0);

    if (solutions.empty()) {
```

```
            os << "No solution found\n";
        } else {
            for (size_t i = 0; i < solutions.size(); ++i) {
                if (solutions.size() > 1) {
                    os << "Solution " << (i + 1) << " of " << solutions.size() << ":\n";
                } else {
                    os << "Solution:\n";
                }
                printBoard(solutions[i], os);
                if (i + 1 < solutions.size()) {
                    os << "\n";
                }
            }
        }
    }
};

int main() {
    const std::vector<std::string> filenames = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (const auto& filename : filenames) {
        std::cout << "========================================\n";
        std::cout << "File: " << filename << "\n";
        std::cout << "========================================\n";

        SudokuSolver solver;
        if (!solver.loadFromFile(filename)) {
            std::cout << "Error: Could not open or read " << filename << "\n\n";
            continue;
```

```
        }

        std::cout << "Original Puzzle:\n";
        solver.printOriginal();
        std::cout << "\n";

        solver.solveAndDisplay();
        std::cout << "\n";
    }

    return 0;
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the "GenAI Code comparison" lecture.

Correctness: Both ChatGPT and Gemini generated programs that use classes and recursive backtracking to solve Sudoku puzzles. Both check the rows, columns, and 3x3 boxes to make sure the numbers follow Sudoku rules. ChatGPT checks for invalid characters while reading the file, while Gemini skips characters it doesn't recognize. Both programs work with the given puzzles, but testing them with incorrect inputs would help make sure they handle errors properly.

Execution time: Both programs use backtracking, which means they try different numbers until they find a solution. This can take longer for difficult puzzles. ChatGPT looks for the next empty cell, while Gemini goes through the board in order. Both try numbers from 1 to 9. Since I didn't measure their actual running times, I can't say which one is faster.

Space complexity: Both programs use a 9x9 grid and recursion, so they don't need much memory for the basic solving process. The main difference is that ChatGPT prints each solution as soon as it finds it, while Gemini stores all solutions in a vector before printing them. This means Gemini can use more memory when a puzzle has multiple solutions.

Maintainability: Both programs are fairly easy to understand because they organize the code into classes and methods. ChatGPT's code is shorter and simpler, but it could use more comments. Gemini has better comments and uses std::array, but some parts are more complicated, especially where it temporarily changes the board during validation.

f. Select one to use as the basis for this assignment and justify why you picked it.

I decided to use ChatGPT's code as the starting point because I found it simpler and easier to understand. It also prints solutions directly instead of storing all of them, which helps save memory. Another reason I chose it is that it checks for invalid characters in the input files. Gemini's code was also good, but I felt ChatGPT's version would be easier for me to improve and maintain.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: I added a check for extra content after the 81 expected puzzle cells, preserving the initial-clue conflict validation. This makes malformed input easier to detect.

Execution time: I improved how the program chooses empty cells. Instead of always picking the first empty cell, it now looks for the cell with the fewest possible numbers. This helps reduce unnecessary guesses and can make the program faster, especially for harder puzzles.

Space complexity: I kept the feature that prints solutions immediately instead of storing them in memory. The program still uses a 9x9 grid and recursion, so it doesn't require much extra memory.

Maintainability: I added more comments to explain what the different parts of the code do. I also made the variable names clearer and organized the explanations so the program would be easier to read, understand, and modify later.